

Celery at the center of your pipelines

Celery is a Burden. WPipe is the Zen. 🧘🚀

The context

Distributed tasks shouldn't require a PhD in RabbitMQ/Redis configuration. If you're building a pipeline, why manage a message broker? WPipe: The Zen of Pythonic Orchestration.

Why engineers shift to wpipe

- No Broker Needed: SQLite handles the state.
- @state Decorator: Transform any function into a resilient pipeline step in 1 line.
- < 50MB RAM: Efficiency that Celery workers can only dream of.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=1)
def job(data):
    return {"result": worker(data["task"])}

pipe = Pipeline(pipeline_name="brokerless")
pipe.set_steps([job])

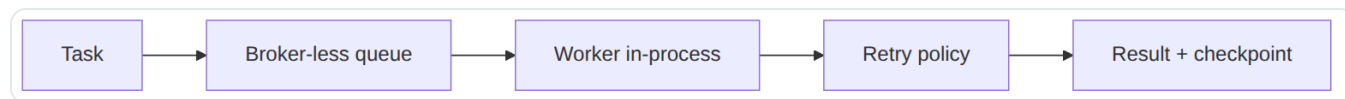
# Async-safe, but no Redis/RabbitMQ required.
```

At a glance

Aspect	Legacy / Celery	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Celery** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.